UNIVERSIDADE DO VALE DO ITAJAÍ

ESCOLA POLITÉCNICA

Curso de Ciência da Computação – Campus Kobrasol

Teoria da Computação

RELATÓRIO TÉCNICO: OTIMIZAÇÃO DO ALGORITMO PASSEIO DO CAVALO UTILIZANDO TÉCNICA PARBERRY

Ana Clara Kniss

Mariana Ferreira

Melissa Moreira de Oliveira

São José, 2026

1. INTRODUÇÃO

O Problema do Passeio do Cavalo consiste em encontrar uma sequência de movimentos válidos do cavalo em um tabuleiro de xadrez de forma que todas as casas sejam visitadas exatamente uma vez. Quando o percurso termina em uma casa que possui um movimento válido para a casa inicial, o passeio é denominado fechado.

Esse problema é amplamente estudado na área de algoritmos e teoria dos grafos, pois pode ser modelado como a busca de um caminho ou ciclo Hamiltoniano em um grafo. Diversas abordagens foram propostas ao longo dos anos, desde algoritmos baseados em busca exaustiva até métodos heurísticos e construtivos capazes de gerar soluções de forma eficiente.

Neste trabalho são analisadas duas abordagens para a resolução do problema: uma solução baseada em *backtracking* combinada com a heurística de Warnsdorff e uma implementação do algoritmo de dividir e conquistar proposto por Ian Parberry. O objetivo é comparar as características dessas abordagens e avaliar seu comportamento na geração de passeios válidos para diferentes tamanhos de tabuleiro.

2. MODELAGEM DO PROBLEMA

O tabuleiro é modelado como um grafo não direcionado, no qual cada casa corresponde a um vértice e cada movimento válido do cavalo corresponde a uma aresta entre dois vértices.

Um passeio do cavalo corresponde a uma sequência de vértices visitados sem repetição. Como o objetivo é visitar todas as casas exatamente uma vez, a solução do problema equivale à busca de um caminho Hamiltoniano no grafo. Quando o passeio retorna à casa inicial por meio de um movimento válido do cavalo, a solução corresponde a um ciclo Hamiltoniano.

Para um tabuleiro de dimensão n×n, o grafo possui n2 vértices. Cada vértice pode possuir até oito arestas, correspondentes aos movimentos possíveis do cavalo. As restrições do problema exigem que todas as casas sejam visitadas exatamente uma vez e que nenhum movimento realizado viole as regras de deslocamento da peça.

As duas abordagens implementadas utilizam representações distintas para a solução. A versão baseada em *backtracking* utiliza uma matriz para armazenar a ordem de visita das casas, enquanto a implementação baseada no algoritmo de Parberry representa o passeio como um ciclo em um grafo, armazenando para cada casa suas duas vizinhas no percurso.

2.1. TIPO DO PROBLEMA

O Problema do Passeio do Cavalo é um problema combinatório pertencente à área de teoria dos grafos. Sua formulação está diretamente relacionada ao problema do caminho Hamiltoniano e, no caso dos passeios fechados, ao problema do ciclo Hamiltoniano.

A solução baseada em *backtracking* caracteriza-se como um algoritmo de busca em espaço de estados, explorando diferentes sequências de movimentos até encontrar uma solução válida. A heurística de Warnsdorff é utilizada para reduzir o número de alternativas exploradas, priorizando movimentos que levam a casas com menor quantidade de opções futuras.

Já o algoritmo de Ian Parberry caracteriza-se como uma abordagem construtiva baseada em dividir e conquistar. Em vez de realizar buscas sobre o espaço de soluções, o algoritmo constrói diretamente um passeio fechado a partir da combinação de soluções menores previamente conhecidas.

Dessa forma, o trabalho contempla duas classes distintas de algoritmos para o mesmo problema: uma abordagem baseada em busca com retrocesso e uma abordagem construtiva recursiva baseada em decomposição do tabuleiro. Isso permite analisar diferenças de desempenho, escalabilidade e aplicabilidade entre os métodos.

3. ABORDAGENS

As soluções usadas para resolver o Problema do Passeio do Cavalo são: **busca recursiva com** *backtracking*, combinada com a **heurística de Warnsdorff,** e o algoritmo de dividir e conquistar publicado por Ian Parberry.

3.1 BACKTRACKING + HEURÍSTICA DE WARNSDORFF

Nessa solução, o tabuleiro é representado por uma matriz de inteiros, em que cada posição armazena a ordem em que foi visitada pelo cavalo, enquanto casas não visitadas permanecem com valor -1.

A cada passo, o algoritmo gera todos os movimentos possíveis a partir da posição atual e os ordena conforme a quantidade de movimentos futuros disponíveis. Dessa forma, prioriza-se visitar primeiro as casas com menor grau de liberdade, reduzindo a chance de bloqueio em etapas posteriores. Caso um caminho não leve à solução completa, o algoritmo desfaz a jogada realizada e tenta uma nova alternativa (*backtracking*).

3.2 DIVIDIR E CONQUISTAR DE IAN PARBERRY

Essa solução é construída montando o tabuleiro a partir de blocos menores, o algoritmo produz um passeio fechado, no qual a última casa volta à primeira por um salto do cavalo, tendo como restrições de tamanho do tabuleiro maior que 6 e de lados pares.

A representação do passeio é feita como um grafo, no qual cada casa guarda quais são as suas 2 vizinhas dentro do passeio, pois em um passeio fechado toda casa é ligada com exatamente 2 outras, de modo a formar um único ciclo.

O tabuleiro é dividido em 4 quadrantes, sendo construído de modo recursivo. Cada lado é partido em 2 metades pares, essa divisão só para quando os quadrantes ficam pequenos o bastante (lado entre 6 e 10), sendo esses, caso-base.

Para cada caso-base, já existe um passeio fechado pronto, no artigo original esses passeios ficam computados previamente e dispostos durante a execução do algoritmo, em nossa implementação, eles são obtidos por uma busca que exige que o ciclo tenha certas arestas fixas perto dos cantos, e depois, ficam guardados em cache para reuso.

Após isso, é feita a união de cada ciclo (quadrante). No ponto de encontro dos quadrantes, o algoritmo remove uma aresta de cada quadrante e a substitui por arestas cruzadas que ligam os quadrantes entre si, de modo a fundir os 4 ciclos em um só ciclo que percorre o tabuleiro inteiro.

Devido ao fato de construir a solução diretamente sem fazer testes e desfazer caminhos(backtracking), o algoritmo é linear ao número de casas.

4. DETALHAMENTO DA IMPLEMENTAÇÃO ESCOLHIDA

A implementação do algoritmo é feita na classe **PasseioParberry**. O passeio é mantido como um grafo não direcionado numa lista de adjacência **adj**, em que **adj[c]** guarda as casas ligadas à casa **c** dentro do passeio.

Cada casa é identificada por um índice único via **id (x,y) = x*n + y**, e a conversão inversa é feita por coordX e coordY. Sobre essa estrutura operam 4 funções: **adicionarAresta**, **removerAresta** e **existeAresta**, **salto** (a verificação, que decide se duas casas estão a um movimento de cavalo de distância).

A construção é feita na função recursiva **construir (x0, y0, largura, altura)**, que aplica o dividir e conquistar. Se o menor lado do retângulo for <= 10, trata-se de um caso-base e o bloco recebe um passeio fechado pronto (via **colocarBase**), caso contrário, cada lado é partido em duas metades pares pela função metade, gerando quatro quadrantes. A função então se chama recursivamente para os quatro e, ao final, une os ciclos resultantes com três chamadas a **unirRetangulos**.

Os blocos-base são obtidos **por tourBaseEstruturada(largura, altura)**. Em vez de embutir os passeios como constantes (como faz o artigo), a função procura um passeio fechado

para o bloco por meio de uma busca em profundidade aleatorizada (Warnsdorff com desempate e corte de becos sem saída), aceitando apenas um ciclo que contenha as oito arestas estruturadas exigidas nos cantos, pois são essas mesmas que garantem que a costura no centro sempre encontre as ligações necessárias. Para evitar recomputação, o resultado é memorizado num cache estático (**cacheBase**), compartilhado por todas as instâncias: assim, cada tamanho de bloco é calculado uma única vez durante toda a execução. A função **colocarBase** então traduz a sequência de casas do bloco em arestas no grafo global, deslocadas para a posição (x0, y0) do quadrante.

Depois de cada quadrante ter o seu próprio passeio fechado, restam quatro ciclos separados que precisam virar um só. Essa fusão é feita pela função **unirRetangulos**, que junta dois ciclos vizinhos de cada vez, usando a troca de arestas descrita pelo artigo.

Em um passeio, cada casa está ligada a exatamente duas vizinhas. A função então olha para uma ligação em cada ciclo: as casas **u e un** (vizinhas no ciclo de um bloco) e as casas **v e vn** (vizinhas no ciclo do bloco ao lado). Para a troca funcionar, ela procura, na fronteira entre os dois blocos, um arranjo dessas quatro casas em que u e v estejam a um salto de cavalo e **un e vn** também, pois são essas as novas ligações que serão criadas, e elas têm de ser movimentos válidos. Encontrado o arranjo, a função remove as duas ligações antigas (**u–un e v–vn**) e adiciona as duas cruzadas (**u–v e un–vn**).

O efeito disso é fundir os dois ciclos. Ao cortar **u–un**, o primeiro ciclo deixa de ser fechado e vira um "fio" com pontas **u e un,** ao cortar **v–vn**, o segundo vira outro fio com pontas **v e vn**. As duas ligações novas amarram as pontas de forma cruzada, costurando os dois fios num único ciclo. Como cada casa apenas troca uma das suas duas ligações por outra (ninguém fica com mais nem com menos que duas), o passeio continua válido. Em cada nível da recursão, três dessas junções transformam os quatro ciclos dos quadrantes num único ciclo.

A função iniciar primeiro zera o grafo de adjacência e chama **construir(0, 0, n, n)**, que monta, pela divisão recursiva e pelas junções, um único ciclo cobrindo o tabuleiro inteiro. Em seguida, percorre esse ciclo a partir da casa pedida pelo usuário, saltando de vizinho em vizinho até voltar ao começo.

5. ANÁLISE DE COMPLEXIDADE

A análise de complexidade das abordagens implementadas evidencia diferenças significativas quanto à forma de obtenção da solução para o Problema do Passeio do Cavalo.

5.1 Backtracking com Heurística de Warnsdorff

A solução baseada em *backtracking* explora o espaço de estados do problema por meio de busca recursiva. A cada posição visitada, o algoritmo gera os movimentos válidos do cavalo e seleciona aqueles que ainda não foram percorridos. A heurística de Warnsdorff reduz a quantidade de estados explorados ao priorizar casas com menor número de movimentos futuros possíveis, diminuindo a ocorrência de becos sem saída.

Apesar dessa otimização, a abordagem continua pertencendo à classe dos algoritmos de busca, cujo pior caso permanece exponencial. Em situações desfavoráveis, o algoritmo pode necessitar explorar um grande número de caminhos alternativos antes de encontrar uma solução válida.

Assim, sua complexidade temporal pode ser representada por:

$$O(b^d)$$

onde *b* apresenta o fator médico de ramificação e *d* corresponde ao número de casas do tabuleiro.

Em relação ao consumo de memória, o algoritmo utiliza uma matriz para armazenar a ordem de visita das casas e uma pilha de chamadas recursivas proporcional ao tamanho do passeio, resultando em complexidade espacial:

$$O(n^2)$$

5.2 Algoritmo de Ian Parberry

A abordagem proposta por Ian Parberry utiliza a técnica de dividir e conquistar para construir diretamente um passeio fechado.

O tabuleiro é subdividido recursivamente em quatro quadrantes até atingir blocos-base de dimensões reduzidas. Cada bloco recebe um passeio fechado previamente calculado e, posteriormente, os ciclos são unidos por meio de operações locais de substituição de arestas.

Como cada casa participa de um número constante de operações durante a construção da solução, o tempo de execução cresce proporcionalmente ao número de vértices do grafo.

A recorrência associada ao algoritmo pode ser descrita por:

$$T(n) = 4T\left(\frac{n}{2}\right) + O(1)$$

Aplicando o Teorema Mestre, obtém-se:

$$T(n) = O(n^2)$$

5.3 Comparação Teórica

A Tabela abaixo resume as características assintóticas das duas abordagens.

Algoritmo	Complexidade Temporal	Complexidade Espacial
Backtracking + Warnsdorff	Exponencial (pior caso)	$O(n^2)$
Parberry	$O(n^2)$	$O(n^2)$

Embora ambas as abordagens utilizem memória proporcional ao tamanho do tabuleiro, o algoritmo de Parberry apresenta melhor escalabilidade teórica por construir diretamente a solução, sem necessidade de explorar caminhos alternativos.

6. RESULTADOS EXPERIMENTAIS

Os experimentos foram realizados em tabuleiros quadrados com dimensões variando de 8×8 até 100×100 casas. Foram avaliados o tempo médio de execução, o consumo de memória de pilha (stack), o consumo de heap e a memória RAM utilizada durante a execução.

A Tabela 1 apresenta os resultados obtidos para ambas as implementações, enquanto as Figuras 1, 2, 3 e 4 mostram a evolução do tempo de execução e do consumo de memória em função do tamanho do tabuleiro.

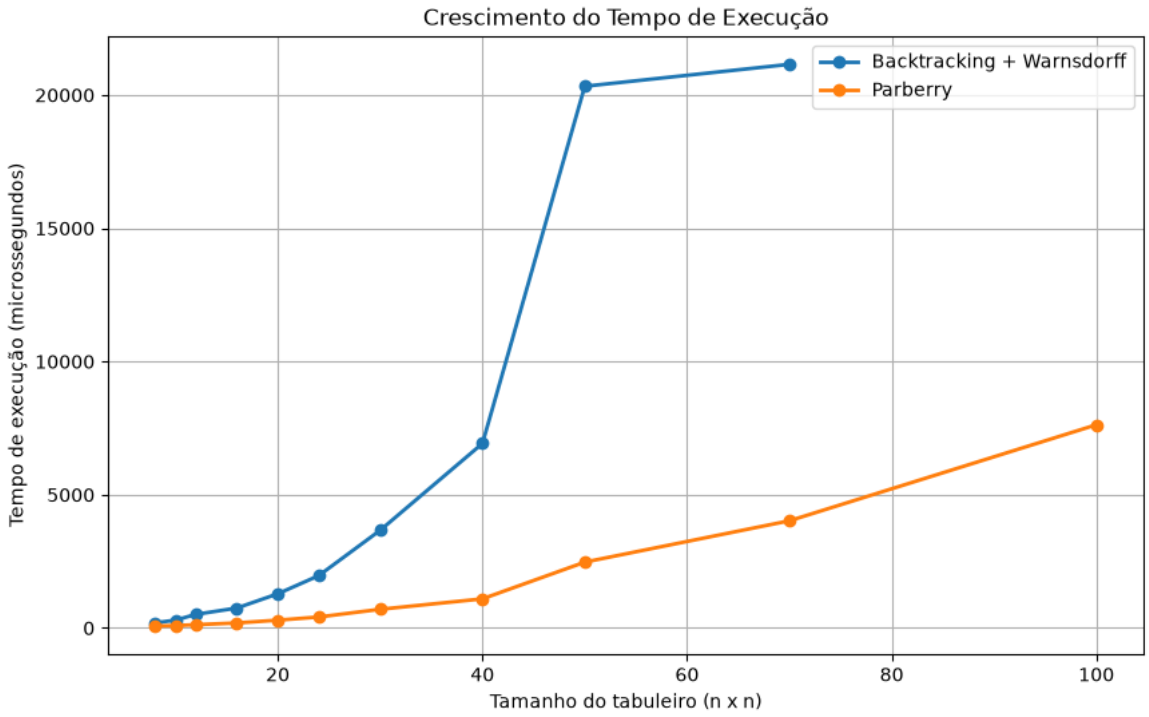
6.1 Tempo de Execução

A Tabela 1 apresenta os resultados obtidos para o tempo médio de execução dos algoritmos avaliados.

Tabela 1 – Resultados experimentais obtidos para diferentes tamanhos de tabuleiro.

n	Algoritmo	Tempo (µs)	Stack (KB)	Heap (KB)	RAM (MB)
8	Backtracking	192,2	11,2	3,3	5,01
8	Parberry	73,9	43,3	4,5	5,10
10	Backtracking	294,8	65,8	7,6	5,12
10	Parberry	89,1	25,0	12,0	5,14
20	Backtracking	1283,4	69,0	24,6	5,27
20	Parberry	293,7	3,1	30,6	5,28
40	Backtracking	6924,5	275,2	97,9	5,51
40	Parberry	1093,8	3,2	122,2	5,52
70	Backtracking	21151,0	842,4	381,7	6,25
70	Parberry	4023,4	53,6	441,7	6,54
100	Backtracking	SKIP	-	-	-
100	Parberry	7625,8	3,5	889,7	6,42

Figura 1 – Crescimento do tempo de execução em função do tamanho do tabuleiro.

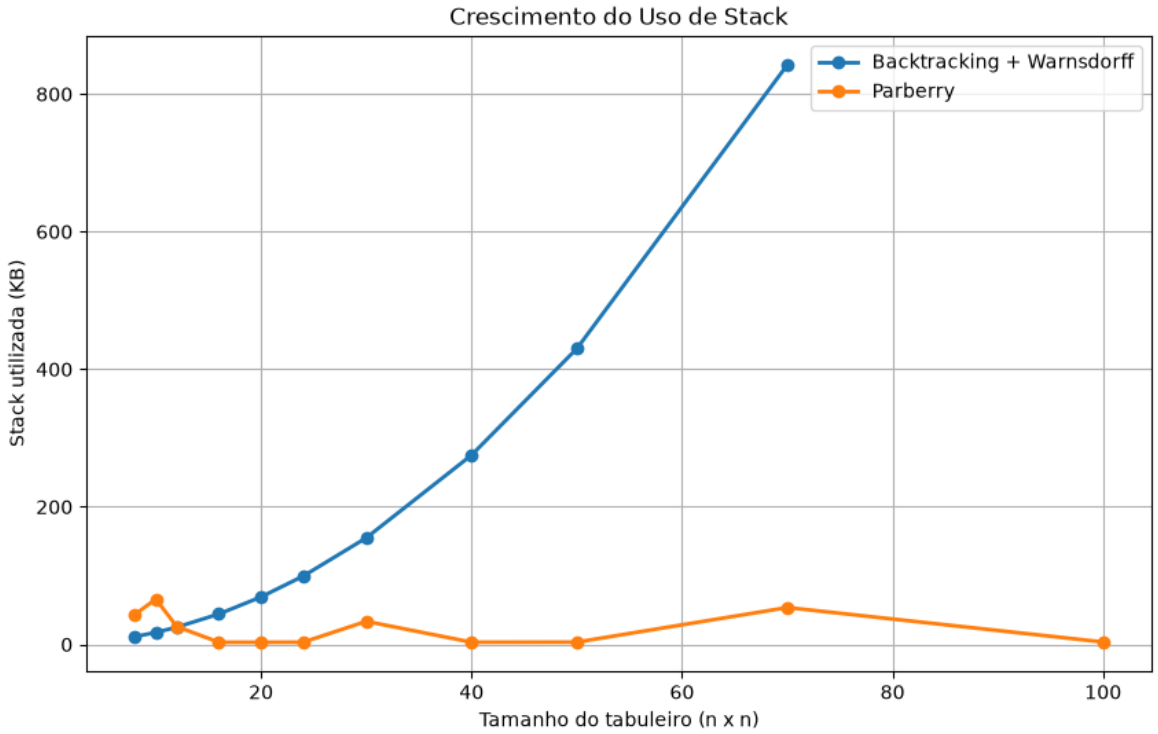


Conforme observado na Figura 1, ambos os algoritmos apresentam aumento no tempo de execução à medida que o tamanho do tabuleiro cresce. Entretanto, a implementação baseada na técnica de Parberry mantém tempos significativamente inferiores aos observados na abordagem de Backtracking com heurística de Warnsdorff. A diferença torna-se mais evidente em tabuleiros de maior dimensão, evidenciando a melhor escalabilidade da estratégia de dividir e conquistar.

6.2 Consumo de Memória

6.2.1 Uso de Stack

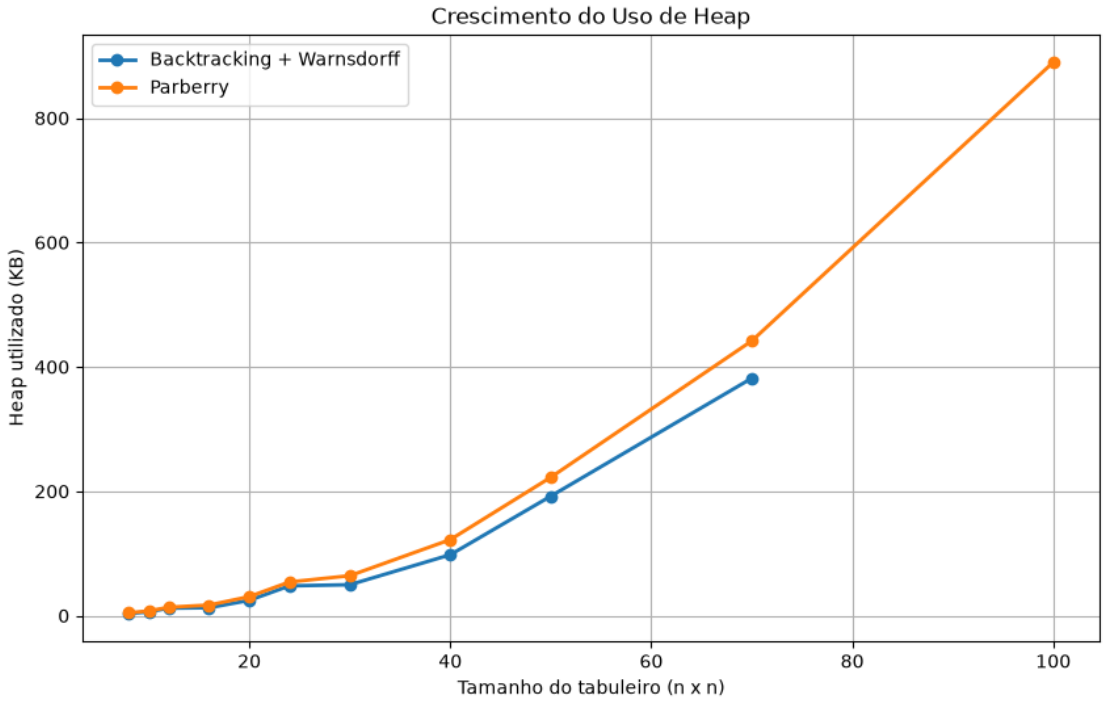
Figura 2 – Consumo de memória *stack* em função do tamanho do tabuleiro.



A Figura 2 mostra o crescimento do consumo de memória *stack* para ambas as implementações. Observa-se que o algoritmo baseado em Backtracking apresenta crescimento mais acentuado devido à profundidade da recursão utilizada durante a busca.

6.2.2 Uso de Heap

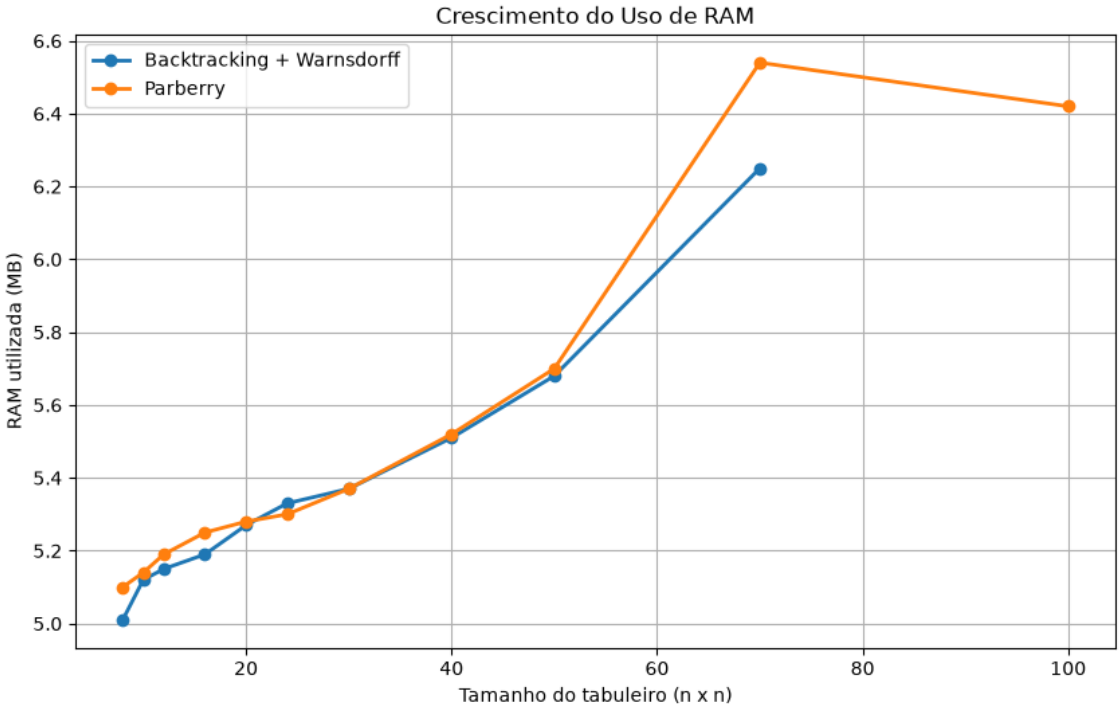
Figura 3 – Consumo de memória heap em função do tamanho do tabuleiro.



Conforme ilustrado na Figura 3, o algoritmo de Parberry utiliza maior quantidade de memória *heap* para tabuleiros maiores, consequência da representação do passeio por meio de estruturas de adjacência e do armazenamento dos ciclos intermediários.

6.2.3 Uso de RAM

Figura 4 – Consumo total de memória RAM em função do tamanho do tabuleiro.



A Figura 4 demonstra que o consumo total de memória RAM permaneceu relativamente estável em ambas as abordagens, variando entre aproximadamente 5 MB e 6,5 MB ao longo dos experimentos.

7. REFERÊNCIAS

PARBERRY, Ian. **An Efficient Algorithm for the Knight's Tour Problem**. *Discrete Applied Mathematics*, v. 73, n. 3, p. 251–260, 1997. Disponível em: https://www.cs.auckland.ac.nz/~mcw/Teaching/320/refs/divide-conquer/knights-tour.pdf. Acesso em: 19 jun. 2026.

RUSSELL, Stuart; NORVIG, Peter. **Solving problems by searching**. In: RUSSELL, Stuart; NORVIG, Peter. *Artificial Intelligence: A Modern Approach*. 3. ed. Upper Saddle River: Prentice Hall, 2010. cap. 3, p. 64-119.